

Evolving nopCommerce for

Multi-Brand Enterprise Autonomy

UA - AS UC
Ilker Atik - 123947

The Strategic Challenge: Scenario A



Enterprise Cohesion

Creating a shared digital foundation and unified customer discovery experience across multiple acquisitions.



Local Autonomy

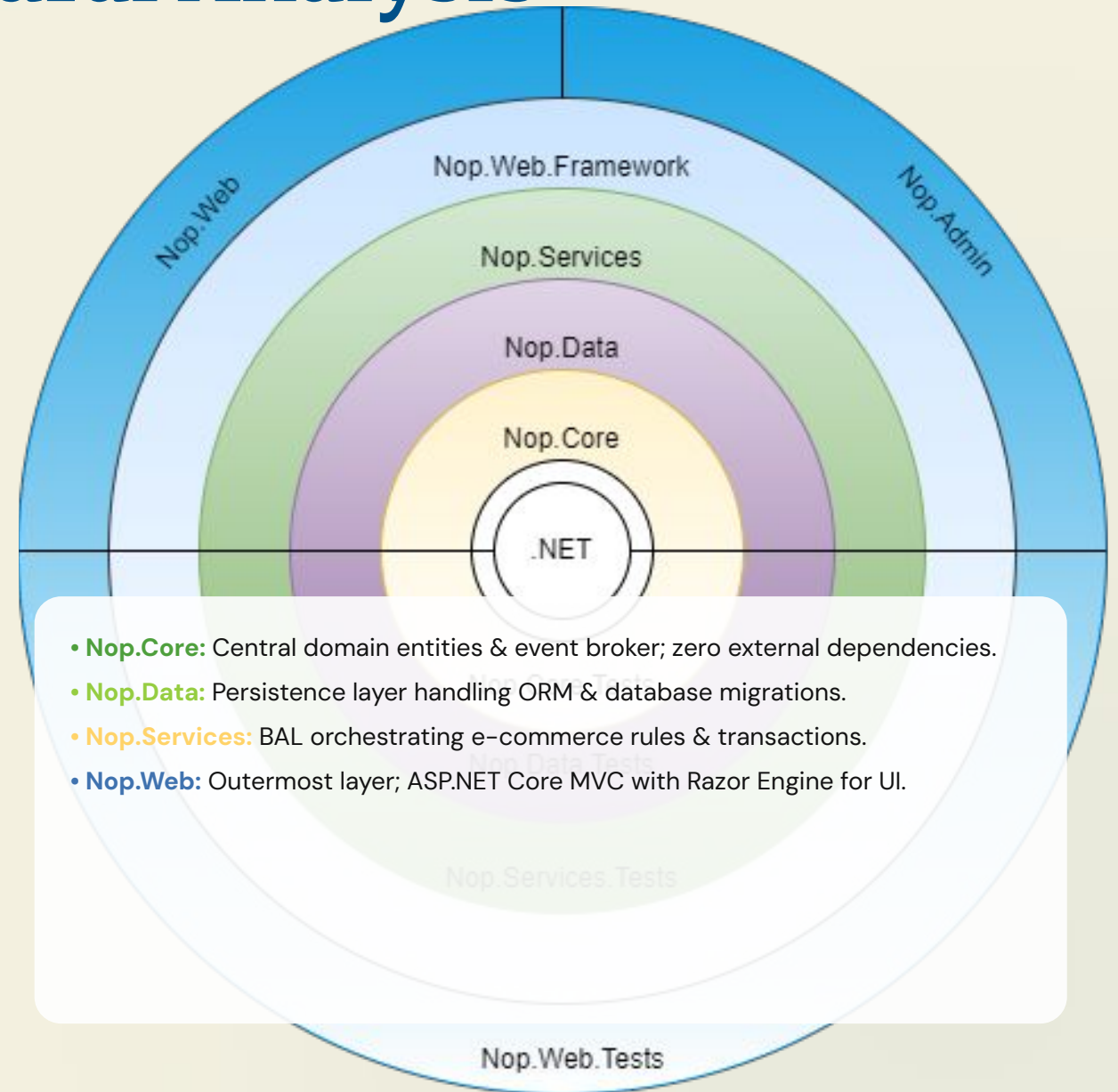
Preserving unique brand catalogs, pricing logic, and ERP rhythms for each business unit (BU).

Conflict: Native multi-store (shared DB) violates strict fault-isolation requirements.

Current-State Architectural Analysis

Architectural Foundations

- **Production-Grade Modular Monolith:** Built on ASP.NET Core & .NET 9.
- **Onion Architecture:** Strict inward dependency flow toward the core domain.
- **Initialization Mechanics:** NopEngine (IEngine) bootstrapper uses ITypeFinder for dynamic assembly scanning.
- **Decoupled Extensibility:** INopStartup interface allows isolated modules to register dependencies via Autofac DI.
- **Cross-Platform:** Supports Windows, Linux, macOS, and Docker.



Architectural Drivers (ADD)



Fault Containment

BU failure must not crash the group experience.



Decentralization

Prohibition of shared database shortcuts across boundaries.



Async Decoupling

Event-driven choreography over brittle sync APIs.

Functional Subdomain Decomposition

CORE



Order & Catalog Management

The heart of the business providing competitive advantage. Localized taxonomies, brand storytelling, and complex pricing justify long-term bespoke development. Highly volatile and essential for brand differentiation.

SUPPORTING



Order Fulfillment & Logistics

Necessary for operation but offers no unique market advantage. Covers warehouse routing, picking, and carrier coordination. Secondary to core commerce innovation efforts.

SUPPORTING



Federated Discovery

A shared foundation across the holding group. Provides a unified product discovery and cross-catalog browsing experience spanning multiple business units to satisfy strategic cohesion.

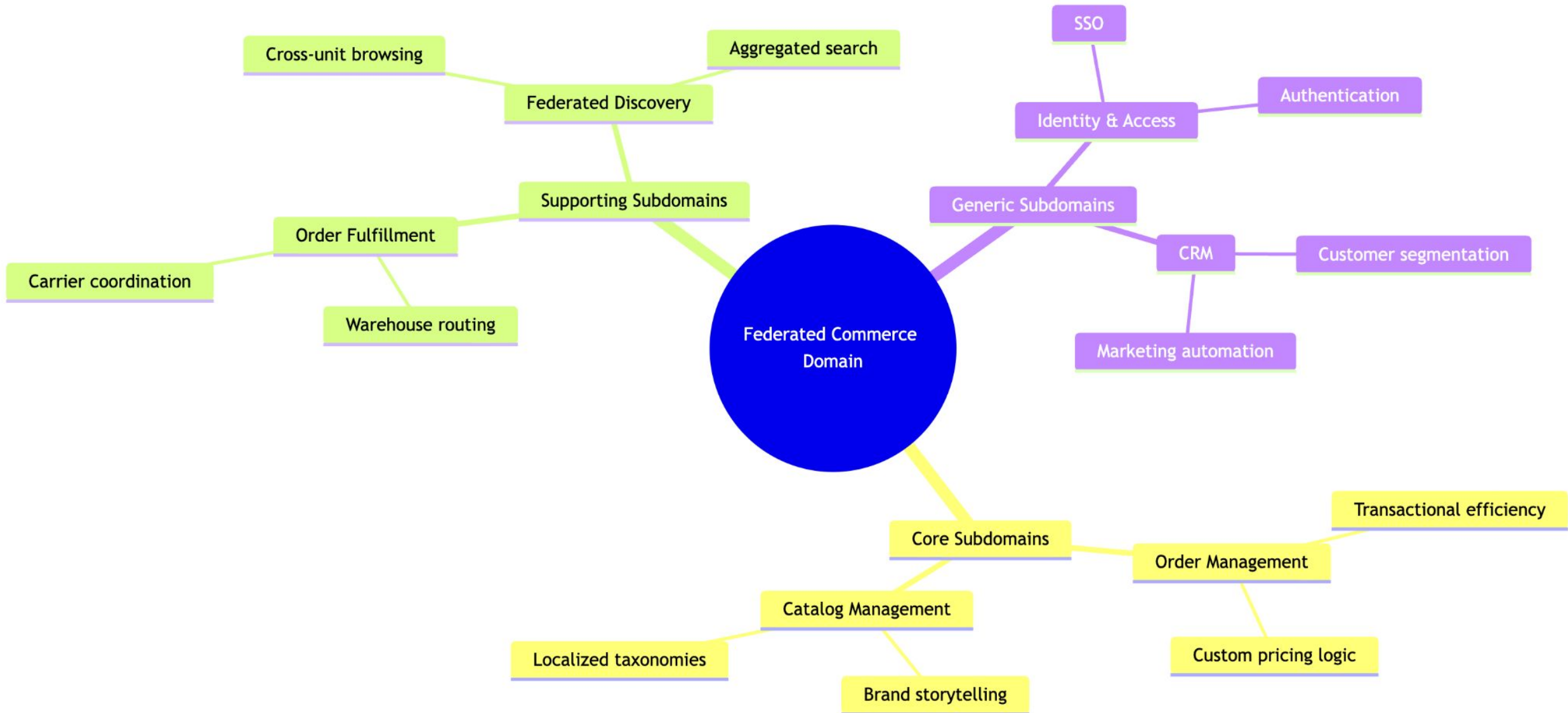
GENERIC



Identity & Relationship

Commodity necessities typically solved with off-the-shelf software. Manages SSO, purchase histories, and marketing automation. Focuses on delegation and authentication standards.

Domains



Bounded Context Definition & System Allocation

CORE



BU Commerce Context

SYSTEM ALLOCATION

Independent ASP.NET Core runtimes;
isolated PostgreSQL nodes

Responsibilities: Commercial brain; orchestration of order capture, localized pricing, and checkout.

Justification: Preserves local brand autonomy; prevents group-wide collapse from local failures.

SUPPORTING



BU Fulfillment Context

SYSTEM ALLOCATION

Localized ERP systems (Odoo) per
Business Unit

Responsibilities: Picking and packing; views Customer as shipping destination and weight constraint.

Justification: Protects storefront performance from resource-heavy warehouse tasks.

SUPPORTING



Group Discovery Context

SYSTEM ALLOCATION

Centralized Search Infra (Meilisearch)

Responsibilities: Read-only index aggregating catalog payloads from all independent units.

Justification: Enables shared browsing without cross-database SQL queries.

GENERIC



Identity & Profile Context

SYSTEM ALLOCATION

Shared CRM (EspoCRM) and Identity Provider (Keycloak)

Responsibilities: SSO management; long-term cross-brand interaction tracking.

Justification: Isolates shared identity from commerce engine transactional noise.

Transactional Outbox for Explicit Reliability

The "Dual-Write" Problem Solution

Maintaining data consistency across isolated contexts requires solving the dual-write failure. If a database update succeeds but the message broker call fails, the system becomes inconsistent.

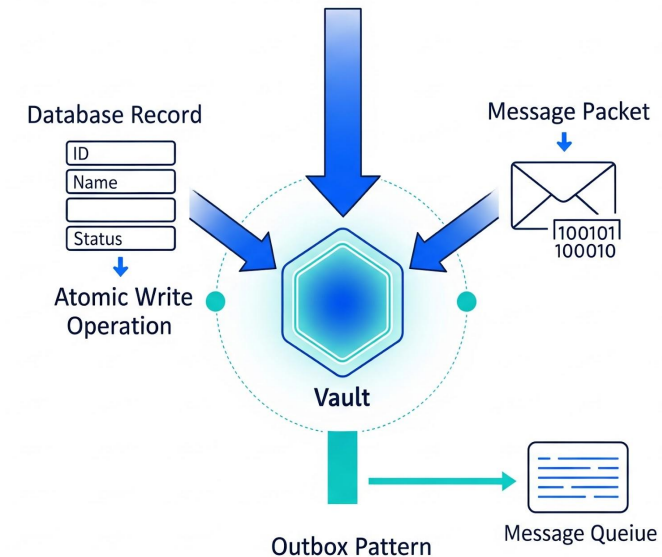
THE ARCHITECTURAL CHOICE

Standardizing on the **Application-Level Transactional Outbox Pattern** over CDC tools like Debezium to minimize infrastructure bloat.

NOPCOMMERCE INTEGRATION

Leverages native **IEventPublisher** and a custom **IConsumer<T>** plugin to intercept lifecycle events.

Writes local state changes and event payloads into a dedicated SQL table within the same **ACID transaction**, guaranteeing atomicity.



Outcome: At-Least-Once Delivery

A lightweight asynchronous relay task reads unpublished records and pushes them to Apache Kafka, insulating the storefront from external dependencies while ensuring delivery.

Quality Attribute Scenarios for Federated Commerce

QA SCENARIO 1

Fault Containment

- **STIMULUS:** Peak traffic order placement while downstream ERP is unavailable.
- **RESPONSE:** Transactional Outbox commits order + event payload atomically to local PostgreSQL.
- **MEASURE:** Checkout completes <500ms; zero latency degradation; at-least-once delivery guaranteed.

QA SCENARIO 2

Asynchronous Decoupling

- **STIMULUS:** Massive catalog updates in a newly acquired brand's local system.
- **RESPONSE:** BU broadcasts standardized CatalogUpdatedEvent via Kafka to Discovery Context.
- **MEASURE:** Searchable on unified storefront <2 mins; zero synchronous coordination needed.

QA SCENARIO 3

Workload Scalability

- **STIMULUS:** 10x holiday surge in orders while central CRM encounters latency.
- **RESPONSE:** Kafka acts as enterprise load-leveler, providing back-pressure management.
- **MEASURE:** Storefront performance unaffected; CRM consumes events at safe maximum rate.

Structural Topology & Decentralized Data Ownership

PHYSICAL TOPOLOGY

Modular Monolith Nodes

- Independent ASP.NET Core nopCommerce instances per Business Unit (BU).
- Dedicated PostgreSQL databases ensure strict data isolation.
- Coordinated via Apache Kafka enterprise event bus.
- Avoids "distributed monolith" anti-pattern via bounded contexts.

DATA OWNERSHIP

Bounded Context Authority

- Each context dictates its own persistent data schema.
- BU Commerce: Owns pricing, carts, and transactions.
- BU Fulfillment: Interprets orders purely as picking instructions.
- Prevents semantic corruption across the holding group.

FEDERATED UI

Shared Experience

- Standalone React/Next.js corporate portal for discovery.
- Consumes read-optimized Meilisearch API for aggregation.
- Zero direct queries to isolated transactional BU databases.
- Ensures cross-brand discovery without backend coupling.

ADR 1: Single-Store Deployment over Native Multi-Store Routing

CONTEXT

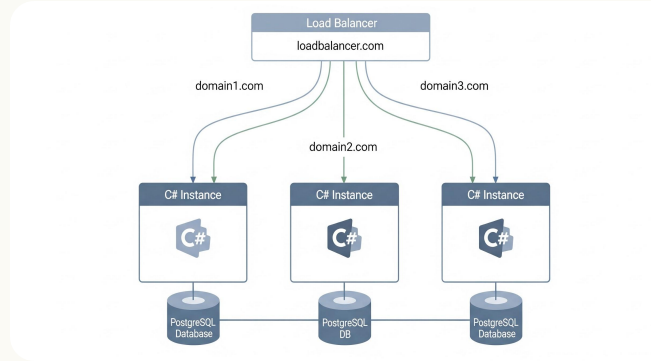
The Shared-DB Shortcut

The platform natively routes multiple domains from one app pool and a shared relational database.

- Dynamic **WebStoreContext** resolution via HTTP HOST headers.
- Shared infrastructure creates high coupling for distinct brand catalogs.

DECISION

Bypassing via Configuration



Prohibit native multi-store logic. Deploy independent runtimes and databases per business unit.

- Maintain source code to avoid technical debt.
- Force **Single-Store context** at deployment.

JUSTIFICATION

Fault Containment

A shared database is a catastrophic single point of failure.

- **Resilience:** Prevents cross-brand resource exhaustion.
- **Autonomy:** Physical isolation allows independent scaling.
- **Evolution:** Adheres to federated guidelines by forbidding shortcuts.

ZERO

Shared Database Reliance

ADR 2: Asynchronous Integration via Transactional Outbox

CONTEXT

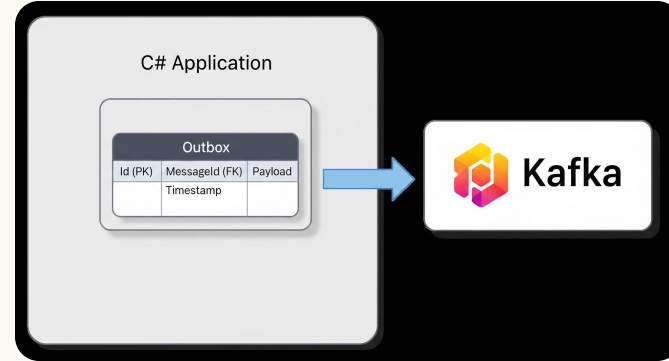
The Dual-Write Anomaly

Transactional data must sync from local instances to shared downstream infra (ERP/CRM) without brittle dependencies.

- Saving to DB and then publishing to a broker is risky.
- If the network call fails after DB commit, systems fall into a **permanently inconsistent state**.

DECISION

Application-Level Outbox



A custom plugin intercepts repository events and persists payloads into a local **Outbox table** within the same SQL transaction.

- A background worker (relay) periodically pushes records to **Apache Kafka**.

JUSTIFICATION

Reliability & Simplicity

Transforms a vulnerable distributed transaction into a **safe local transaction**.

- **Resilience:** Storefront stays functional even if Kafka is down.
- **Guaranteed Delivery:** Securely buffers events for at-least-once delivery.
- **Efficiency:** Avoids infrastructure complexity like CDC/Debezium by using native C# extensibility.

100%

Delivery Guarantee

ADR 3: Centralized Discovery via Meilisearch Asynchronous Views

CONTEXT

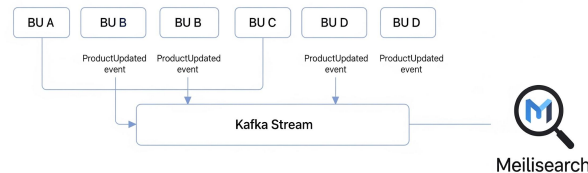
Unified Discovery Needs

To establish a shared foundation, the enterprise requires a unified search experience across all federated business units.

- Executing direct cross-database SQL queries between isolated BUs is **strictly prohibited**.
- Must maintain service boundaries while allowing seamless product browsing.

DECISION

Open-Host Service (OHS)



Deploy a central **Meilisearch** infrastructure. BU contexts publish events to Kafka, which are projected into aggregated materialized views.

- Consumer service builds the search index.
- Queried by Federated Discovery UI.

JUSTIFICATION

Local Autonomy & Speed

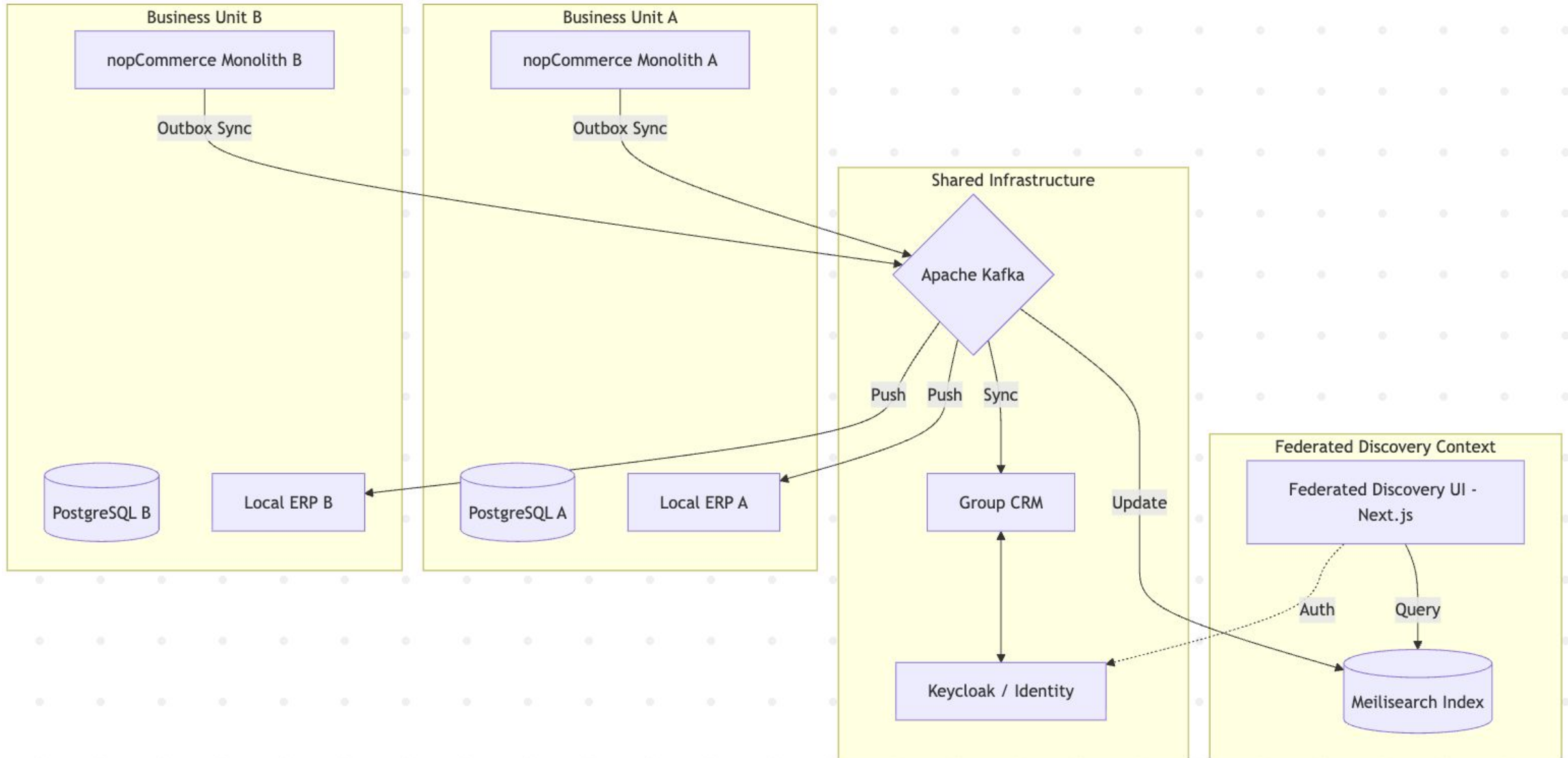
Meilisearch provides lower operational overhead than OpenSearch with superior out-of-the-box relevancy.

- **Autonomy:** BUs manage their own ranking rules and synonyms easily.
- **Trade-off:** Accepts **Eventual Consistency** (approx. 30s) for Absolute Fault Isolation.
- **Resilience:** Central search failures never block local store orders.

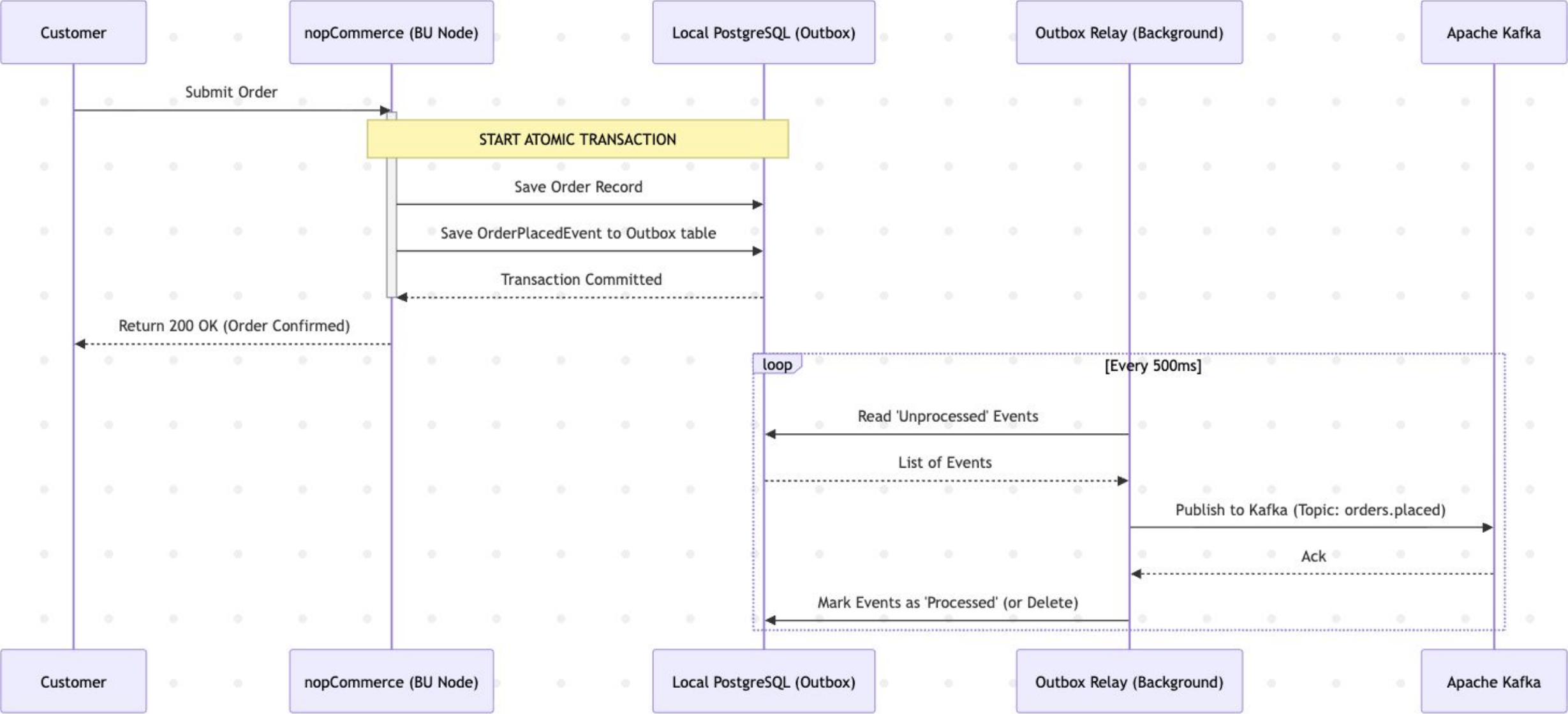
~30s

Consistency Trade-off Window

Event Driven Communication



Order Flow



Target Event-Driven Architecture

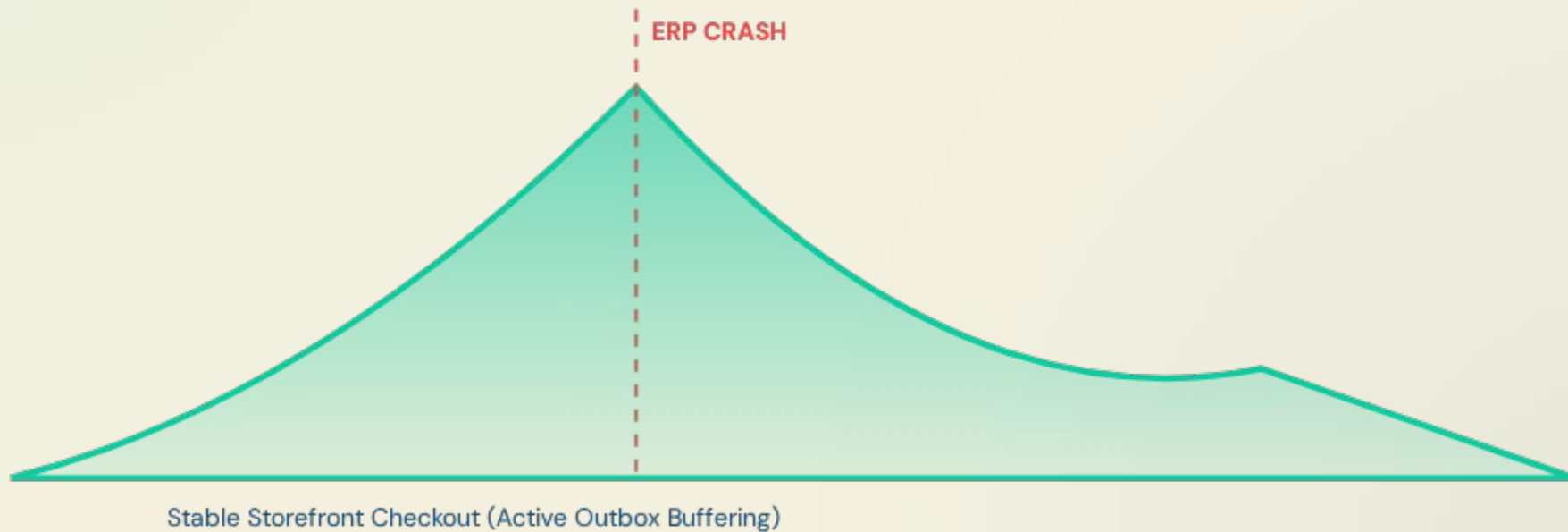
Structural Decentralization:

Transition from a central monolith to physically isolated nodes coordinated via Apache Kafka.

- Isolated BU databases (PostgreSQL)
- Async event-streaming for synchronization
- Materialized views in Meilisearch for high-speed discovery

Resilience: Downstream Failure

Scenario: Local ERP (Fulfillment) Experiences Outage during Sales Spike.



"Local degradation without group collapse" – Storefront continues processing while Outbox buffers the backlog.

Evolution Path & Feasibility

Phase 2

Outbox Plugin & Isolated BU
Nodes.

Phase 1

Kafka & Meilisearch Infra Setup.

Phase 3

Central CRM & SSO Integration.

Q&A